

CV-Based Town Planning Risk System

Team Reference Document — Final Year Project

Predicting Unplanned Urban Growth & Flood Risk from Satellite Imagery using Computer Vision

Purpose of this document:

This document explains the entire project — the problem, the plan, the datasets, the architecture, and the doubts already worked through — so that every teammate can understand it fully without needing prior context.

1. Problem Statement

Indian towns and cities are growing fast, but a large share of this growth is unplanned — buildings coming up on floodplains, water bodies getting encroached, and green cover disappearing. Planners and citizens usually find out about this only after it becomes a crisis (a flood, a dried-up lake, a heat-stressed neighbourhood).

There is currently no easy, accessible tool that watches satellite imagery of a town over time, detects how construction has spread, and flags areas where this growth is risky — before it becomes a disaster.

What We Are Building

A system that:

- Looks at satellite images of a chosen town across multiple years (e.g. 2018 vs 2024)
- Detects how buildings/construction have spread using computer vision (segmentation)
- Combines this with elevation data and water-body locations to flag risky growth (low-lying areas, near water bodies, on lost green cover)
- Shows this as a color-coded risk map (green = safe, yellow = watch, red = high risk)
- Lets a citizen search their area and get a plain-language risk summary

Why This Matters

This is a genuine, underexplored gap. A 2026 systematic review of Computer Vision in urban planning (Marasinghe et al., Sustainable Cities and Society) found that most existing CV-urban-planning research is strong at the "observation" and "analysis" levels (measuring what exists, detecting patterns) but comparatively weak at the "prediction" level (using that data to forecast risk and support real decisions). Our project deliberately targets that predictive gap — this is our core novelty.

2. High-Level Architecture

At a high level, the system has four stages. Each stage takes the output of the previous one.

```
STAGE 1: DATA COLLECTION
  Satellite Imagery (Sentinel-2) + Elevation (SRTM) + Water Bodies (OpenStreetMap)
                                | (all downloaded for the SAME town boundary)
                                v
STAGE 2: OBSERVATION (Computer Vision)
  Segment each satellite image into: Building / Vegetation / Other
  (using a pretrained model, fine-tuned on local images)
                                v
STAGE 3: ANALYSIS (Change Detection)
  Compare Year A vs Year B segmentation maps
  -> Where did NEW construction appear?
  -> Where was green cover LOST?
                                v
STAGE 4: PREDICTION (Risk Scoring -- our novel contribution)
  Combine: new construction + low elevation + proximity to water
  -> Risk Score per zone (Low / Medium / High)
                                v
OUTPUT: Interactive Dashboard + Citizen-Facing Lookup
  - Planner view: full risk map, change-over-time view
  - Citizen view: type an address -> get plain-language risk summary
```

The 5 Levels of "Intelligence" This Project Touches

This framing comes directly from the CV-urban-planning review paper and is useful for explaining your project's depth to your panel:

Level	What it means	Where it appears in our project
Observational	Measuring what's physically there	Segmenting buildings/vegetation from satellite images
Analytical	Detecting patterns & change over time	Comparing Year A vs Year B to find new construction
Correlative	Linking form to outcomes	Linking construction growth to elevation/water proximity
Predictive	Forecasting risk (our novelty)	The Risk Score layer — flagging danger zones
Generative	Suggesting alternatives (out of scope)	Not attempted — noted as future work

Note: we are NOT attempting the "Generative" level (AI redesigning the city) — that is out of scope and would be dishonest to claim. Staying honest about scope is a strength in your viva, not a weakness.

3. Low-Level Implementation Plan

Phase 1 — Environment Setup

- Sign up for Google Earth Engine (free): <https://earthengine.google.com/> (needs Google account + one-time approval)
- Use Google Colab for any model training — free GPU access, no need for expensive hardware
- Install required Python libraries:

```
pip install earthengine-api geemap rasterio geopandas
pip install torch torchvision segmentation-models-pytorch
pip install numpy pandas matplotlib folium streamlit osmnx
```

Phase 2 — Download Satellite Imagery (Sentinel-2)

IMPORTANT: define ONE boundary box for your chosen town and reuse the exact same variable for every download (satellite, elevation, water). This is what keeps all datasets aligned to the same place.

```
import ee, geemap
ee.Authenticate() # opens browser login once
ee.Initialize()

# Define the town boundary ONCE – reuse this everywhere
town_boundary = ee.Geometry.Rectangle([77.55, 12.90, 77.65, 13.00])

def get_image(year):
    collection = (ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
                  .filterDate(f'{year}-01-01', f'{year}-12-31')
                  .filterBounds(town_boundary)
                  .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 10))
                  .median())
    return collection.clip(town_boundary)

img_2018 = get_image(2018)
img_2024 = get_image(2024)

geemap.ee_export_image_to_drive(img_2018, description='town_2018',
                                folder='urban_cv_project', region=town_boundary, scale=10)
geemap.ee_export_image_to_drive(img_2024, description='town_2024',
                                folder='urban_cv_project', region=town_boundary, scale=10)
```

Backup manual sources (if GEE access is delayed): <https://scihub.copernicus.eu/> or <https://earthexplorer.usgs.gov/> (both free, need a free account, allow manual search + download by coordinates/date/cloud-cover).

Phase 3 — Download Elevation Data (SRTM)

```
srtm = ee.Image('USGS/SRTMGL1_003').clip(town_boundary) # SAME boundary as above

geemap.ee_export_image_to_drive(srtm, description='town_elevation',
                                folder='urban_cv_project', region=town_boundary, scale=30)
```

Backup manual source: <https://dwtkns.com/srtm30m/> — pick the tiles covering your town, download directly, no login required.

Phase 4 — Download Water Body / Road Data (OpenStreetMap)

```
import osmnx as ox

water = ox.features_from_place("Your Town Name, India", tags={'natural': 'water'})
water.to_file("water_bodies.geojson", driver="GeoJSON")

roads = ox.graph_from_place("Your Town Name, India", network_type='drive')
```

This is free and requires no manual download portal — the osmnx library pulls the data directly.

Phase 5 — Aligning the Datasets (Resolution Mismatch Fix)

Sentinel-2 images are 10m resolution; SRTM elevation is 30m resolution — different pixel sizes. This is a standard, well-documented GIS step called resampling/reprojection — snap everything to one common grid before combining:

```
import rasterio
from rasterio.warp import reproject, Resampling

with rasterio.open('elevation.tif') as src:
    reproject(
        source=rasterio.band(src, 1),
        destination=aligned_elevation_array,
        resampling=Resampling.bilinear
    )
```

Phase 6 — Building / Green Cover Segmentation (Computer Vision)

Use a pretrained segmentation model (DeepLabV3+) and fine-tune it lightly on local images, since models pretrained on European/US cities don't generalise perfectly to Indian towns.

```
import segmentation_models_pytorch as smp

model = smp.DeepLabV3Plus(
    encoder_name="resnet50",
    encoder_weights="imagenet", # pretrained weights
    in_channels=3,
    classes=3 # building, vegetation, other
)
```

Where to get labeled training data to fine-tune the model:

- Microsoft Building Footprints (India): <https://github.com/microsoft/GlobalMLBuildingFootprints> — free, direct download per country, gives building polygon labels
- Google Open Buildings: <https://sites.research.google/open-buildings/> — free, downloadable by region, also gives building footprints

These solve the "trained on non-Indian cities" limitation flagged in the review paper — use them as ground-truth labels to fine-tune the pretrained model.

Phase 7 — Change Detection

```
import numpy as np

mask_2018 = model.predict(img_2018_tiles)
mask_2024 = model.predict(img_2024_tiles)

# New construction = building in 2024 but NOT building in 2018
new_construction = (mask_2024 == BUILDING_CLASS) & (mask_2018 != BUILDING_CLASS)

# Green cover loss
green_loss = (mask_2018 == VEGETATION_CLASS) & (mask_2024 != VEGETATION_CLASS)
```

Phase 8 — Risk Scoring (Our Novel Contribution)

Keep this rule-based and explainable rather than a black-box model — easier to defend in the viva, and matches best practice from the literature (explainability matters more as decision-relevance increases).

```
import rasterio

with rasterio.open('town_elevation.tif') as src:
    elevation = src.read(1)

def compute_risk(new_construction, elevation, water_distance):
    risk = np.zeros_like(elevation)
    low_elevation = elevation < np.percentile(elevation, 25) # lowest 25%
    near_water = water_distance < 200 # within 200m of water body

    risk[new_construction & low_elevation & near_water] = 3 # HIGH
    risk[new_construction & (low_elevation | near_water)] = 2 # MEDIUM
    risk[new_construction] = 1 # LOW
    return risk
```

Phase 9 — Dashboard (Planner View + Citizen View)

```
# streamlit app
import streamlit as st, folium
from streamlit_folium import st_folium
from geopy.geocoders import Nominatim

st.title("Urban Growth Risk Map")

m = folium.Map(location=[town_lat, town_lon], zoom_start=13)
folium.raster_layers.ImageOverlay(
    image=risk_map_colored, bounds=town_bounds, opacity=0.6
).add_to(m)
st_folium(m, width=800)

# --- Citizen-facing address lookup ---
geolocator = Nominatim(user_agent="urban_risk_app")
user_input = st.text_input("Enter your area/address:")
if user_input:
    location = geolocator.geocode(user_input)
    if not town_boundary.contains(location):
        st.warning("This tool currently covers only [Your Town Name]. "
                  "We're working on expanding coverage.")
    else:
        st.write(get_plain_language_risk_summary(location))
```

Run locally with:

```
streamlit run dashboard.py
```

4. Datasets — Summary & Download Links

Dataset	Purpose	Where to get it	Approx. size
Sentinel-2 imagery	Satellite images, multi-year	Google Earth Engine (via geemap) — earthengine.google.com . Backup: scihub.copernicus.eu / earthexplorer.usgs.gov	~50-150 MB per year per town
SRTM elevation	Elevation for flood-risk logic	Via Earth Engine (USGS/SRTMGL1_003). Backup: dwtkns.com/srtm30m	~5-20 MB
OpenStreetMap water/roads	Water body & road locations	Python osmnx library (features_from_place)	A few MB (vector data)
Microsoft Building Footprints	Ground-truth building labels for fine-tuning	github.com/microsoft/GlobalMLBuildingFootprints	Varies by region
Google Open Buildings	Alternative building footprint labels	sites.research.google/open-buildings	Varies by region

Total realistic storage for ONE town, a few years of imagery: under 3-5 GB. Google Drive's free 15 GB tier is enough — no paid storage needed, as long as we scope to a single town (not multiple cities).

5. Team Roles & Timeline

Suggested Team Split (3-4 people)

Person	Responsibility
Person 1	Data pipeline — Earth Engine downloads, preprocessing, SRTM/OSM integration, dataset alignment
Person 2	Computer Vision model — fine-tune DeepLabV3+, run segmentation, validate accuracy (IoU/F1)
Person 3	Risk scoring logic + change detection analysis
Person 4	Dashboard, citizen-facing lookup, documentation, literature review compilation

Realistic Timeline (5-6 months)

1. Weeks 1-3: Set up Google Earth Engine, download all data, literature review, choose the town
2. Weeks 4-6: Get building-footprint labels (Microsoft/Google datasets), set up training pipeline
3. Weeks 7-10: Fine-tune segmentation model, evaluate accuracy
4. Weeks 11-13: Run change detection between years, sanity-check against known ground truth
5. Weeks 14-16: Build the risk scoring logic, validate against local knowledge
6. Weeks 17-19: Build the dashboard (planner view + citizen view)
7. Weeks 20-22: Testing, polish, write the report, prepare for the viva

6. Doubts Already Raised — and Their Answers

These are real questions raised while planning this project. Read through them — they will likely come up again from teammates or the review panel, so it helps if everyone already knows the answers.

Q1: Will the satellite, elevation, and water datasets actually line up to the same place if we download them separately?

Answer: Yes — as long as we define ONE boundary box (a lat/lon rectangle for our town) and reuse that exact same boundary for every download. The only technical wrinkle is that Sentinel-2 (10m resolution) and SRTM (30m resolution) have different pixel sizes — this is fixed with a standard GIS step called resampling/reprojection (see Phase 5 above), which snaps both datasets onto one common grid before combining them. This is a well-documented, solved problem in GIS — not something we're inventing.

Q2: How much Google Drive storage do we need for the satellite data?

Answer: For a single mid-sized town with a few years of imagery, the total is realistically under 3-5 GB. Google Drive's free 15 GB tier is enough. This assumes we scope the project to ONE town, not multiple cities — which we should, for a final-year project.

Q3: We're just analysing and flagging differences — how does this actually help normal people? It doesn't stop anyone from building somewhere risky.

Answer: This is a fair and important concern. The fix is to make the output ACTIONABLE, not just a heatmap for planners. We add:

- A citizen-facing address/area lookup that gives a plain-language answer (e.g. "This area is at moderate flood risk due to low elevation and recent construction nearby — before buying/building here, consider a soil/drainage assessment").
- A shareable report feature — a resident can forward flagged areas to their local municipal/ward office as evidence, even though our tool has no enforcement authority.
- A "before you build here" advisory mode — check a specific plot BEFORE construction, not just monitor after the fact.

Important: We must be explicit in our report that this is a decision-SUPPORT tool, not a decision-making authority. We cannot legally stop construction — we can only give people and local bodies evidence they didn't have easy access to before. This honesty is a strength in the viva, not a weakness — real CV-based planning tools in published research are framed the same way (informing decisions, not replacing human/regulatory judgement).

Q4: Can the citizen lookup feature work for ANY address in the world?

Answer: No — and it shouldn't try to. Our actual risk analysis (satellite + elevation + building data) is scoped to ONE town only. We handle this cleanly with a boundary check: if a searched address falls outside our covered town, the app returns a clear message like "This tool currently covers only [Town Name]. We're working on expanding coverage," instead of breaking or giving wrong information. The geocoding step itself (turning a typed address into coordinates, via the free Nominatim/OpenStreetMap service) does work worldwide — it's only our underlying risk DATA that is scoped to one town. This is normal — even real production tools launch region-by-region. We present this as intentional scope, not a flaw: the same pipeline is reusable for any new town by re-running the same download-and-train steps.

Q5: Which town should we pick?

Answer: Pick a town/area that gives a strong, honest "why does this matter" story:

- Somewhere with a known flooding or unplanned-construction history in the news — makes the motivation obvious to the panel
- Somewhere a team member knows personally or is near — makes it possible to sanity-check ('ground-truth') whether the model's flags actually make sense
- Not too large — a district/taluk-sized area or a mid-sized town, not an entire major city, to keep the scope manageable

7. Novelty Statement

Use this as the core answer whenever asked "why is this project novel / hasn't this been done before?":

"Existing computer-vision-in-urban-planning research, per a 2026 systematic review (Marasinghe et al., Sustainable Cities and Society), is concentrated at the **observational and analytical** levels — measuring what exists and detecting change patterns. The **predictive** level (using CV outputs to forecast risk and support real decisions) is comparatively underdeveloped and flagged in the literature as higher-stakes and less validated. Additionally, most pretrained CV models are trained on non-Indian urban imagery, raising representativeness concerns for our context. Our project extends CV-based urban analysis to the predictive level specifically for flood/unplanned-growth risk, fine-tuned and validated on Indian town imagery — a combination not directly addressed in prior work."

8. Key Honesty Notes for the Report

- This is a decision-support tool, not a decision-making authority — we do not have regulatory power and should never imply otherwise.
- The risk score is a rule-based, explainable model — not a black-box prediction — this is intentional, for transparency.
- The project is scoped to ONE town as a proof-of-concept; the pipeline is designed to be reusable for other towns, but that expansion is future work, not part of this project's deliverable.
- We are not attempting the "generative" level of CV (AI redesigning the city) — this is explicitly out of scope.

— End of document —